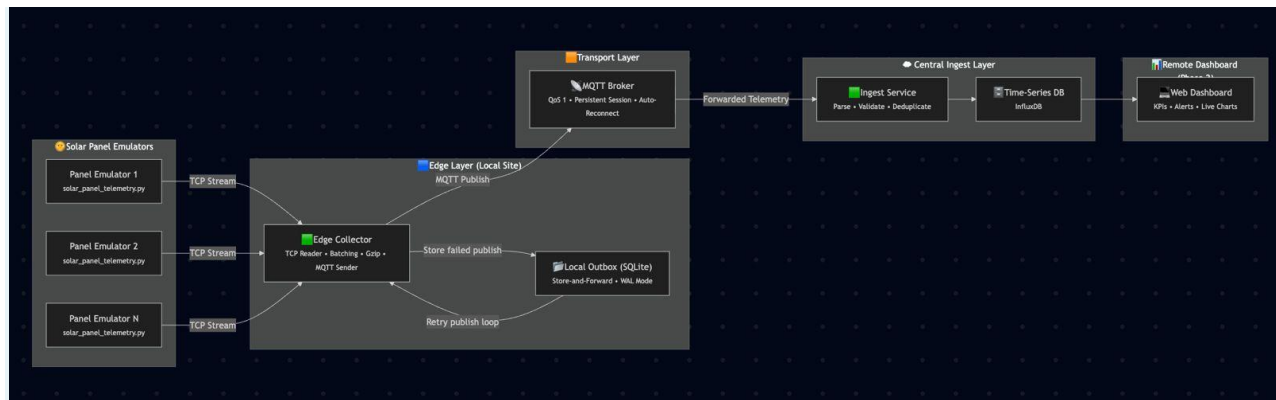


Foxhound Unit

Communication Mechanism & Data Flow



1. Purpose

This document explains the chosen communication mechanism and the end-to-end data flow for the Foxhound Unit project. It describes how the telemetry generator, edge collector, and central ingest interact, the serialization and compression used, persistence strategy, and the validation/deduplication logic.

2. Components (Files & Purpose)

- `src/solar_panel_telemetry.py` — Telemetry generator (CSV or JSONL)
- `src/edge_collector_server.py` — TCP collector; receives lines, validates them, batches, compresses, and publishes
- `src/central_ingest.py` — MQTT subscriber; deduplicates and writes to a datastore or fallback JSONL
- `src/records_handler.py` — Validation/normalization utilities that ensure the correct 13-field schema
- `src/ingest_fallback.jsonl` — Fallback file for ingest when InfluxDB is not configured
- `data/` — Sample files
- `tests/` — Pytest tests validating the flow and components (`test_phase1.py`)

3. Chosen Communication Mechanism — Rationale

Generator → Collector

- **Transport:** TCP (plain text / line-delimited JSONL or CSV)
- **Rationale:** Simplicity and compatibility. The generator can pipe output using netcat (nc) or simple TCP clients to the collector's listening port. This also allows running the generator and collector locally without an MQTT broker.

Collector → Cloud/Backend

- **Transport:** MQTT (Paho MQTT client is used)
- **Rationale:** MQTT provides durable, decoupled pub/sub between the edge and central services. The collector can batch and publish telemetry messages to a topic while the central ingest remains subscribed. MQTT is lightweight and suits intermittent connectivity.

Persistence / Buffering

- SQLite is used as an outbox on the edge (`edge_buffer.db`) to queue compressed publish payloads when MQTT is unavailable. This ensures messages are not lost when network connectivity is intermittent.
- Central ingest uses a deduplication SQLite DB (`ingest_dedup.db`) to prevent duplicate insertions.

Serialization & Compression

- JSONL is the primary message format in the pipeline. The collector compresses batches using gzip before publishing to MQTT to conserve bandwidth.

4. Data Schema & Validation

Each telemetry record follows the final schema of 13 fields:

```
timestamp_utc, panel_id, string_id, status, fault, power_w, voltage_v,  
current_a, irradiance_wm2, ambient_temp_c, cell_temp_c, orientation_deg,  
tilt_deg
```

- **Validation:** The collector uses RecordsHandler (`src/records_handler.py`) to reject lines that do not match the expected schema. CSV input will be converted (header must match required fields).
- **Deduplication:** Central ingest deduplicates messages using the composite of timestamp and panel_id (tests reflect dedup logic). Duplicate messages are dropped.

5. Data Flow (Step-by-Step)

1. Telemetry Generation

The generator (`src/solar_panel_telemetry.py`) produces CSV lines or JSONL lines for configured panels and time windows.

Example:

```
python src/solar_panel_telemetry.py --panels 10 --hours 1 --step 30 --format jsonl
```

2. Push to Collector

The generator streams lines to the collector TCP endpoint, e.g., `| nc localhost 9000`.

Collector receives and buffers incomplete chunks until newline-terminated records are ready. The collector handles message chunking and line re-assembly.

3. Validation & Batching

For each line, RecordsHandler validates the schema. Valid records are appended to the current batch; invalid records are counted and discarded.

The collector batches by count (configurable `batch_size`) or by time (`batch_secs`). When limit reached, it compresses the batch (gzip) and publishes via MQTT.

4. Publish to MQTT

The collector publishes to a topic `panels/{site_id}/telemetry`. The payload is the gzip-compressed JSONL string representing the batch.

Example: `panels/site-01/telemetry`

If MQTT is down or unavailable, the collector persists the compressed batch in the SQLite outbox (`edge_buffer.db`).

5. MQTT Ingest

Central ingest subscribes to the topic(s) and receives compressed batches. It decompresses and parses each JSON line.

For each record the central ingest:

- Deduplicates using `ingest_dedup.db` (if message identical by `panel_id` + timestamp, skip)
- Writes to InfluxDB if configured (by environment variables). If Influx is not available/configured, it writes to `src/ingest_fallback.jsonl`

6. Acknowledgment & Retry

The collector sets stats for messages published and queued.

A background loop sends outbox items when connectivity is restored, ensuring eventual delivery.

6. Failure Modes & Handling

Failure Mode	Handling Strategy
Collector MQTT disconnect	Batches are written to the local SQLite outbox (edge_buffer.db). The send_outbox_loop attempts to retransmit outbox entries once connection is restored.
Invalid records	Rejected and tracked (stats["messages_invalid"] incremented)
Duplicate records on central ingest	Central ingest drops duplicates based on panel_id + timestamp; duplicates are not forwarded to datastore
Parsing and decompression failures	Caught by central ingest and can fallback to ingest_fallback.jsonl

7. Test Summary (Verification)

Tests validate RecordsHandler, collector logic (line handling and batching), outbox flow (push/pop and retries), and central ingest dedup behavior.

Running Tests

Prepare environment & dependencies:

```
python -m venv .venv
./venv/Scripts/Activate.ps1 # Windows PowerShell
pip install -r requirements.txt
pip install pytest
```

Execute tests:

```
pytest -q
```

Results from the Test Run

15 tests passed, 2 warnings

```
v sent batch of 1 messages (213 bytes)
===== short test summary info =====
PASSED tests/test_phase1.py::test_records_handler_valid_json
PASSED tests/test_phase1.py::test_records_handler_csv
PASSED tests/test_phase1.py::test_records_handler_missing_field
PASSED tests/test_phase1.py::test_records_handler_unknown_format
PASSED tests/test_phase1.py::test_edge_add_line_valid
PASSED tests/test_phase1.py::test_edge_add_line_invalid
h_queues
PASSED tests/test_phase1.py::test_outbox_retry
PASSED tests/test_phase1.py::test_handle_client_splits_lines
PASSED tests/test_phase1.py::test_central_fallback
PASSED tests/test_phase1.py::test_central_dedup
PASSED tests/test_phase1.py::test_end_to_end
15 passed, 2 warnings in 1.71s
```

8. How to Run the Minimal Pipeline Locally

1. (Optional) Start an MQTT broker (mosquitto) locally:

```
# e.g. on macOS / Linux:

mosquitto
```

2. Start central ingest (subscribe to MQTT):

```
python src/central_ingest.py --mqtt-host localhost --mqtt-port 1883
```

3. Start edge collector:

```
python src/edge_collector_server.py --mqtt-host localhost --mqtt-port 1883 --site-id
site-01 --port 9000
```

4. Generate and send telemetry:

```
python src/solar_panel_telemetry.py --panels 5 --hours 0.1 --format jsonl | nc
localhost 9000
```

9. Configuration Notes

- Central ingest can write to InfluxDB; configure environment variables `INFLUX_URL`, `INFLUX_TOKEN`, `INFLUX_ORG`, `INFLUX_BUCKET` to enable.
- Default fallback sink if Influx not configured: `src/ingest_fallback.jsonl`
- Default outbox DB path on the collector: `edge_buffer.db`. Default dedup DB path on ingest: `ingest_dedup.db`. Change these in the module or via environment/config as desired.

10. Data Security & Privacy

- Telemetry data used here is synthetic and for demonstration/testing.
- Do not commit secrets; use environment variables to pass credentials for production services like InfluxDB.

11. Limitations & Observations

- Telemetry deduplication is based on `panel_id` + timestamp only (adjust if more robust checks needed).
- Exact message ordering and concurrency interleaving varies — tests are deterministic where necessary by the test harness and the generator can be seeded for deterministic outputs if needed.
- For guaranteed exactly-once semantics across unreliable networks, further measures (transactional storage and idempotent insertion) would be needed.

12. Appendix: Useful Commands

Run unit tests:

```
pytest
```

Generate CSV:

```
python src/solar_panel_telemetry.py --panels 100 --hours 1 --step 60 --out
telemetry.csv
```

Start collector listening on port 9000 and publish to MQTT:

```
python src/edge_collector_server.py --mqtt-host localhost --mqtt-port 1883 --site-id  
site-01 --port 9000
```

Run central ingest:

```
python src/central_ingest.py --mqtt-host localhost --mqtt-port 1883
```
